Glasgow Traceroute

A Rust Implementation of Paris Traceroute

by

Ryan Bramwell

Registration Number: 202216892

Bachelor of Science

CS408: Individual Project

Except where explicitly stated, all the work in this report, including appendices, is my own and was carried out during my final/fourth year. It has not been submitted for assessment in any other context.

Computer and Information Sciences

March, 2026

Abstract

Traceroute is widely used by network engineers to measure network paths, but it produces inaccurate results in load-balanced networks. Paris Traceroute addresses this limitation, while the Multipath Detection Algorithm (MDA) extends its functionality to discover multiple paths. However, most existing implementations are typically written in C or C++, which rely on manual memory management and can introduce a range of vulnerabilities.

This project presents Glasgow Traceroute, a memory-safe implementation of Ping, Paris Traceroute, and a simplified MDA in Rust. It constructs raw ICMP and UDP probes while maintaining control over packet header fields to ensure correct behaviour in load-balanced networks. The implementation was evaluated using a software-defined network environment, where controlled topologies were used to verify accuracy.

The results demonstrate that a memory-safe implementation of these tools is not only possible but can be reliably validated in a controlled environment, while maintaining accurate and consistent path measurement.

Acknowledgements

I would like to thank my supervisor, Dr Daniel Thomas, for his support and guidance throughout the project.

I would also like to thank my girlfriend, Paloma, for her continued support and faith in everything I do.

Finally, I would like to thank my dad for supporting me throughout my studies, my mum and sister for their love and encouragement, and my uncle for getting me interested in technology during my school days.

Contents

1 Introduction **1**
1.1 Motivation . . . 1
1.2 Aim . . . 1
1.3 Objectives . . . 1
1.4 Report Structure . . . 2

2 Background Literature **3**
2.1 Internet Protocols Used in Network Measurement . . . 3
2.2 Ping . . . 4
2.3 Classic Traceroute . . . 4
2.4 Traceroute and Load Balanced Networks . . . 5
2.5 Paris Traceroute . . . 5
2.6 Multipath Detection Algorithm (MDA) . . . 7
2.7 Dublin Traceroute . . . 8
2.8 Memory Safety and Rust . . . 8

3 Specification & Design **10**
3.1 Methodology . . . 10
3.2 Analysis . . . 11
3.3 Requirements . . . 11
3.4 Functional and Non-Functional Requirements . . . 11
3.5 Design . . . 13

4 Product **18**
4.1 Implementation Overview . . . 18
4.2 Packet Handling . . . 18
4.3 Probing . . . 19
4.4 Interface . . . 22
4.5 Verification & Validation . . . 22

5 Results & Evaluation **25**
5.1 Results of Evaluation . . . 25

6 Conclusion **29**
6.1 Interpreting the Results . . . 29
6.2 Challenges and Limitations . . . 29
6.3 Future Work . . . 30

A Appendix A - Detailed Specification and Design **32**

1 Introduction

1.1 Motivation

As modern software continues to rely on highly distributed networks for communication, accurate and reliable network measurement tools remain indispensable. Such utilities are typically written in C and C++, reflecting the systems-level control needed when operating close to the network stack. While these implementations have proven effective over decades of use, the scale and complexity of the resulting codebases can make reasoning about their behaviour challenging. These systems-level trade-offs are apparent in tools handling externally generated data, where explicit memory management is necessary to limit unintended behaviour and potential vulnerabilities. Such vulnerabilities include buffer overflows, where data exceeds the bounds of allocated memory and overwrites adjacent regions.

Considering these challenges, modern systems programming languages like Rust offer stronger guarantees around memory safety without sacrificing low-level control. One such tool that stands to benefit from this approach is Paris Traceroute, which builds upon traditional Traceroute by improving the accuracy of path discovery in load balanced networks.

1.2 Aim

The aim of this project is to develop an extensible, Rust-based implementation of Paris Traceroute that can accurately discover network paths in load-balanced environments while overcoming the memory-safety concerns present in earlier versions.

1.3 Objectives

The project aim is realised through the following five objectives:

- Understand how Paris Traceroute addresses the limits of traditional Traceroute, particularly in relation to multipath routing and load balancing.
- Analyse how network protocol headers are structured, how responses are matched to outgoing packets and which fields must remain constant in the presence of load balancing.
- Apply memory safe Rust programming practices to implement Ping, Paris Traceroute and

Multipath Detection Algorithm (MDA), keeping the code modular and isolating operations that handle external network data to maintain memory safety.

- Evaluate the accuracy of each tool by validating its measurements against known topologies in a software-defined network, comparing observed paths against the expected routing behaviour.
- Create and document the implementation as a reusable library with clear documentation and interfaces to support further research of other client applications.

1.4 Report Structure

The structure of this report is as follows:

- **Chapter 2** reviews the background literature and research relevant to network measurement including Classic Traceroute, Paris Traceroute and the Multipath Detection Algorithm (MDA).
- **Chapter 3** outlines the methodology, project specifications and the system architecture of Glasgow Traceroute.
- **Chapter 4** describes the product that is Glasgow Traceroute and delves in to the specifics of its technical implementation.
- **Chapter 5** evaluates the results of the project and assesses the extent to which the stated objectives were achieved.
- **Chapter 6** reflects on the development process and technical challenges, discusses the limitations of the final product, outlines potential future work, and concludes the dissertation.

2 Background Literature

This section details the background research that has facilitated the development of this project. It highlights the importance of network path measurement and how it has evolved in response to increasingly complex routing, showing how each tool was developed to address the limitations of its predecessor.

2.1 Internet Protocols Used in Network Measurement

Network measurement tools rely on Internet protocols operating at the network and transport layers of the Open Systems Interconnection (OSI) model [1] to characterise network behaviour and topology. The key protocols used in network measurement are summarised below:

- **Internet Protocol (IP):** The Internet Protocol's main purpose is to transport datagrams across a network [2]. Within the IP header, the source and destination fields enable routers to correctly forward packets toward their intended destination. Packet forwarding occurs hop-by-hop and is controlled in part by the Time To Live (TTL) field. TTL was originally intended to prevent packets from becoming stuck in indefinite loops by limiting the number of hops a packet can traverse. However, it has become particularly useful in allowing network tools to deliberately trigger responses from intermediate routers.
- **Internet Control Message Protocol (ICMP):** The Internet Control Message Protocol (ICMP) is used to report errors and provide diagnostic information related to transmitted IP packets [3]. When a router discards a packet due to the Time To Live (TTL) field reaching zero, it returns an ICMP Time Exceeded message to the sender. ICMP also defines Echo Request and Echo Reply messages, which are used by diagnostic tools to test network reachability.
- **User Datagram Protocol (UDP) and Transmission Control Protocol (TCP):** Transport layer protocols UDP and TCP are encapsulated within IP datagrams and provide applications the ability to communicate between hosts. TCP offers reliable communication through connection establishment [4], while UDP provides a fast connectionless alternative with minimal overhead[5].

The OSI model provides a cohesive layered architecture in which each layer contributes to the transmission and routing of packets across interconnected computer networks. These protocol

interactions form the basis of the network measurement tools discussed in the following section.

2.2 Ping

Developed in 1983 by Mike Muuss, Ping is still one of the most widely used tools for testing network connectivity [6]. Derived from the protocols summarised above, Ping measures the round trip time from a host to destination using IP/ICMP ECHO_REQUEST and ECHO_REPLY packets.

While Ping provides valuable diagnostic information about communication between hosts, it offers no insight into the intermediate devices through which packets travel. This limitation propelled the need for a tool capable of revealing the path taken by packets across the network.

2.3 Classic Traceroute

Jacobson's Traceroute laid the foundation for network path discovery and has been a standard method for route measurement since its introduction in 1989 [7]. Like the earlier diagnostic tool Ping, Traceroute relies on behaviour defined in ICMP and IP to provoke diagnostic responses from remote routers. To understand how Traceroute achieves path discovery, it is necessary to consider the role of the Time To Live (TTL) field in the IP header. TTL controls how many hops a packet may traverse before being discarded.

Jacobson observed that when a packet's TTL expires, the router discarding the packet is required to return an ICMP Time Exceeded message to the sender. By deliberately sending probe packets with progressively increasing TTL values, Traceroute is able to trigger responses from each router along the path, allowing the sequence of hops between a source and destination to be inferred. This is illustrated below in Figure 1.

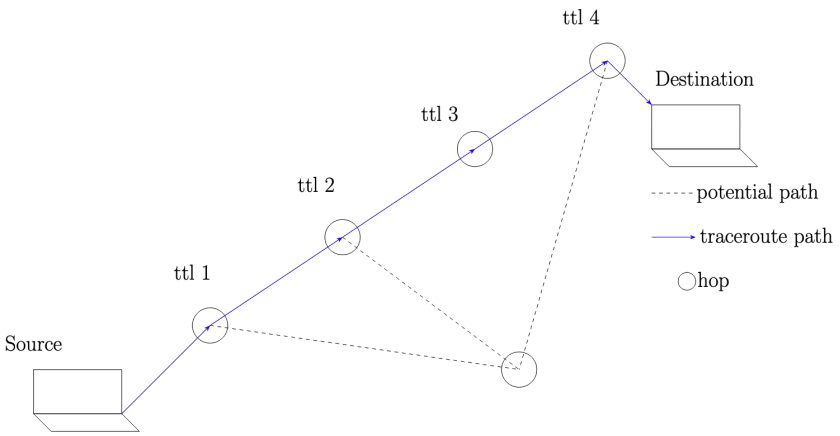

Figure 1: Traceroute hop discovery using incrementing TTL values. The solid path shows the measured route, while the dashed lines indicate alternative paths not taken

2.4 Traceroute and Load Balanced Networks

To understand the limitations of Jacobson's Traceroute, it is necessary to consider the concept of load balancing. In networking, load balancing refers to the distribution of network traffic across multiple equal-cost paths available to a router. Routing protocols such as Open Shortest Path First (OSPF) [8] use Dijkstra's Algorithm to determine a router's set of equal-cost next-hop paths. Once these paths have been computed, routers may distribute traffic across them using a variety of load-balancing strategies, including per-packet, per-destination, and per-flow approaches . This work focuses on per-flow load balancing. In this approach, the router computes a hash over selected packet header fields, producing what is known as a flow identifier that ensures packets belonging to the same flow follow the same path [9].

This presents a problem for Traceroute as header fields are intentionally changed to match ICMP responses with the probes that triggered them, causing each probe to have a unique flow identifier. This results in each probe being load balanced down potentially different paths which can cause measurements to return incorrect or confusing paths that do not correspond to any real route through the network. This is illustrated below in Figure 2.

2.5 Paris Traceroute

To mitigate incorrect and confusing paths caused by load balanced networks, Augustin et al. introduced Paris Traceroute [9]. Their solution ensures that probes maintain a constant flow identifier so that intermediate routers performing per-flow load balancing compute the same

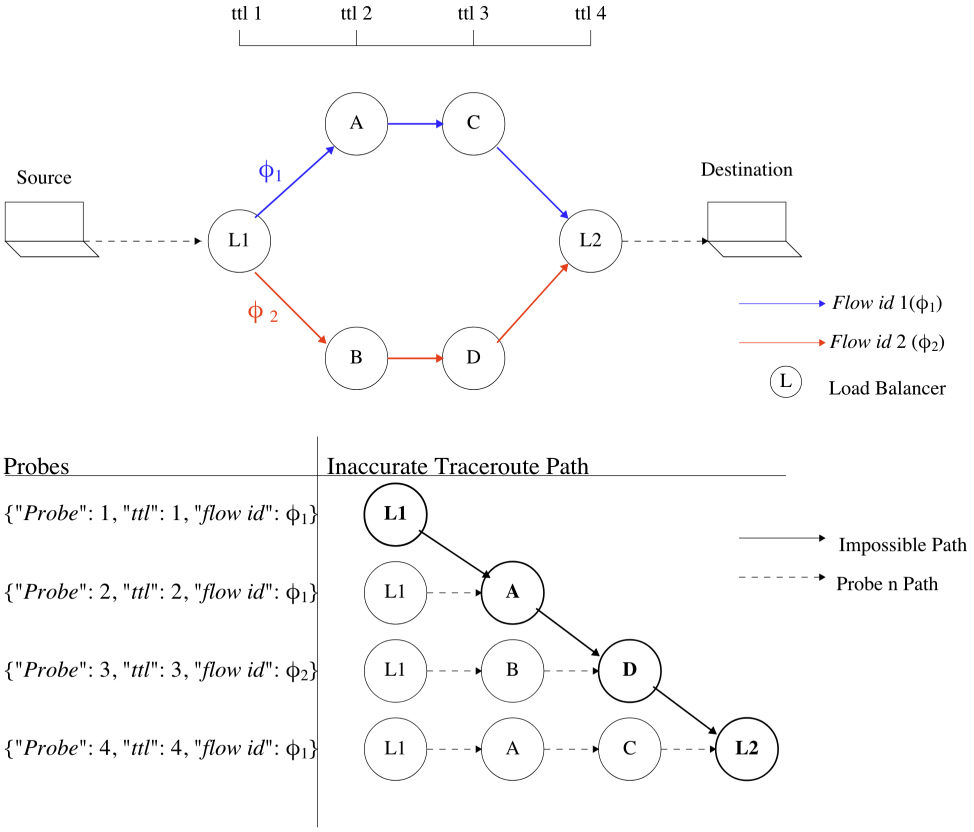

Figure 2: Traceroute measurement artefact caused by per-flow load balancing, where probes from different flows are combined into a non-existent path. Adapted from [9]

hash, resulting in all probes following an identical path. To achieve this while still allowing responses to be matched with the probes that generated them, Paris Traceroute varies specific header fields depending on the probe protocol. The relevant fields for each protocol are summarised in Table 1.

Protocol	Varied by Classic Traceroute	Varied by Paris Traceroute	Flow Hash Fields
IP	None	None	Type of Service, Protocol, Source IP, Destination IP
UDP	Destination Port, Checksum	Checksum	Source Port, Destination Port
ICMP Echo	Sequence Number, Checksum	Sequence Number and Identifier varied	Code, Checksum
TCP	None	Sequence Number	Source Port, Destination Port

Table 1: Comparison of header field manipulation in classic Traceroute and Paris Traceroute. Adapted from [9]

While Paris Traceroute successfully identifies a consistent path by controlling packet flow, it can reveal only a single path. Augustin et al. note that a complete route tracing tool should be capable of describing all paths that different flows may take, not just one. This limitation incited the development of an algorithm that can explore every path in a load balanced network with statistical certainty. It is this algorithm, the Multipath Detection Algorithm (MDA), that the next section examines.

2.6 Multipath Detection Algorithm (MDA)

The Multipath Detection Algorithm (MDA), also introduced by Augustin et al. (2007), extends the capabilities of Paris Traceroute by addressing its primary limitation of only revealing a single path between a source and destination [10]. In networks employing load balancing, multiple valid paths may exist simultaneously, and accurately characterising these paths requires exploring the set of routes that different flows may take.

MDA achieves this by sending probes with varying flow identifiers in order to explore the set of possible paths. To determine when probing should stop, the algorithm uses a statistically derived stopping rule that estimates the probability that an undiscovered interface remains at a given hop. Probing continues until this probability falls below a predefined confidence threshold. The required number of probes depends on the number of interfaces already observed at a hop, with values for a 95% confidence level.

Although MDA enables Paris Traceroute to discover multiple paths with statistical accuracy, its algorithm is considerably complex. Furthermore, as with Paris Traceroute, the implementation

of these techniques was developed in C++, a language that relies on manual memory management. Manual memory allocation introduces potential vulnerabilities and increases implementation complexity. These challenges are examined in the next section and have motivated more recent implementations.

2.7 Dublin Traceroute

One such implementation is Dublin Traceroute developed by Andrea Barberio which was built to provide a fast NAT-aware multipath Traceroute tool without memory leaks or crashes [11]. Its NAT-detection feature builds upon Paris Traceroute by recognising if packet responses have been modified by Network Address Translation devices.

In addition, Dublin Traceroute's multipath discovery strategy has a more practical approach. Instead of using Probability thresholds like MDA, the tool allows the user to specify the number of paths to explore.

Despite the practical advancements, Dublin Traceroute is still primarily written in C++ further highlighting the need for a memory safe implementation. The next section examines how modern systems programming languages like Rust can provide stronger memory safety guarantees while still offering low level control for network packet construction.

2.8 Memory Safety and Rust

Network programming requires low-level control in order to construct raw networking packets and interact with operating system networking interfaces. Developers often opt for languages like C and C++ that offer explicit low level control of hardware. However, these languages rely on manual memory management, which introduces a range of vulnerabilities such as buffer overflows, use-after free bugs and null pointer dereferences. These common vulnerabilities are significant; the Microsoft Security Response Center estimates that approximately 70% of assigned CVEs are related to memory safety [12]. Although modern C++ provides mitigations such as Resource Acquisition is Initialisation (RAII) and smart pointers, the language itself still facilitates undefined behaviour leaving the responsibility largely with the developer. This highlights the need for a memory safe systems programming language like Rust.

Rust is a relatively new programming language that aims to combine strong memory safety guarantees with performance comparable to that of traditional alternatives. As well as verifying

memory safety at compile time through it's ownership model, Rust provides a mechanism to deal with operations that can't be statically verified such as interacting with raw packet buffers. This is achieved through the use of Rust's *unsafe* blocks that isolate such operations without exposing them to the rest of the program.

Rust also excels in performance, achieving results comparable to traditional systems programming languages such as C and C++ while outperforming other memory safe languages like Go [13]. Bugden and Alahmar demonstrate this through a series of benchmarking experiments measuring CPU time and memory usage across several prominent programming languages. Their results show that Rust consistently performs near the top across these metrics, indicating that strong memory safety guarantees can be achieved without sacrificing execution performance. An example of these results is shown in Figure 3.

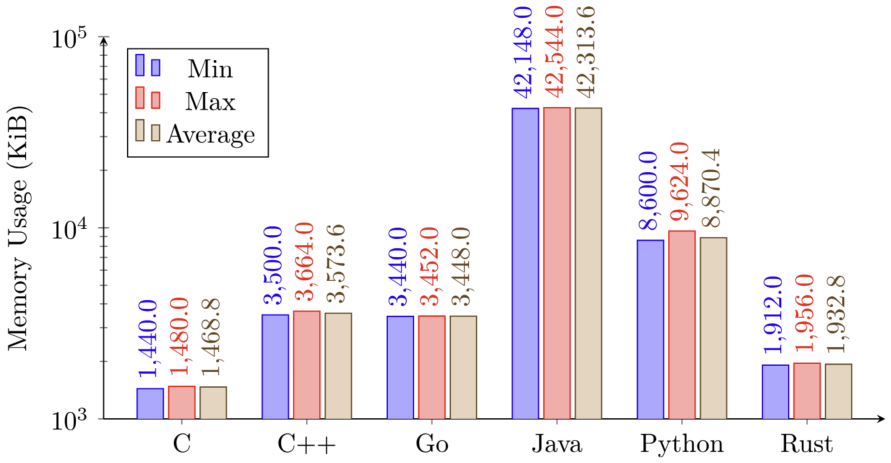

Figure 3: Figure from [13] showing Comparison of Monte Carlo Pi estimation memory usage using SimpleRNG (lower is better)

3 Specification & Design

This chapter details the project specification and methodology that guided the completion of the product implementation, as well as the functional and non-functional requirements that informed each design decision.

3.1 Methodology

An iterative development approach, akin to the Agile methodology, was used to incrementally develop minimal viable products at each stage of the project's lifecycle. Due to the nature of systems programming development, each stage was validated before progressing to the next, ensuring that low-level networking functionality behaved as expected before introducing additional complexity. This method also allowed for a development process that closely mirrored the progression of the literature review where each application was built upon its predecessor. This process is illustrated in Figure 4, where the loop between the construction and evaluation phases represents the iterative validation of each implementation stage.

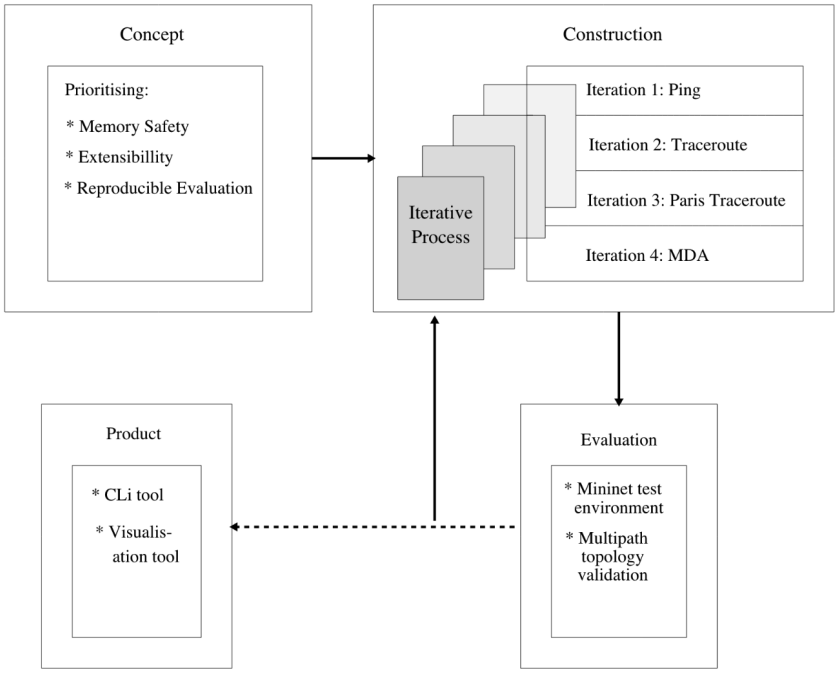

Figure 4: Iterative Development Process for Glasgow Traceroute Measurement Tool. Adapted from Agile Lifecycle

3.2 Analysis

The decision to develop Glasgow Traceroute was motivated by the need for an extensible, testable, and memory-safe implementation of Paris Traceroute, as identified through the literature review. Existing implementations demonstrate the effectiveness of Paris Traceroute and its multipath extensions, but they are typically written in languages such as C or C++, which rely on manual memory management and can introduce reliability and security concerns. Developing a Rust-based implementation therefore provides an opportunity to explore how modern memory-safe systems programming techniques can be applied to network measurement tools.

The intended behaviour and specification of the software artifact to be developed are outlined in the following section in the form of system requirements.

3.3 Requirements

3.4 Functional and Non-Functional Requirements

- **User Story:** "As a user, I want to be able to ping a host and receive a statistical summary of the probe results."

  FUNCTIONAL REQUIREMENTS

  The system must:

  - build ICMP and UDP probe packets encapsulated within valid IPv4 headers, with control over relevant header fields.
  - send probes using raw sockets, parse ICMP responses correctly and report packet loss and echo replies.

  NON-FUNCTIONAL REQUIREMENTS

  The system must:

  - handle socket timeouts seamlessly, displaying error messages accordingly.
  - return results via a structured interface so callers can present them as they like.

- **User Story:** "As a user, I want to be able to Traceroute a network ensuring all probes follow the same flow in load balanced networks"

FUNCTIONAL REQUIREMENTS

The system must:

- discover routes by sending probes with incrementing TTL, keeping flow identifiers constant across hops (Paris Traceroute).
- avoid classic Traceroute artefacts through consistent probe fields.

NON-FUNCTIONAL REQUIREMENTS

The system must:

- Demonstrate robustness in scenarios where Paris Traceroute is known to crash, or hang.

- **User Story:** "As a user, I want to discover multiple valid paths to a destination in load balanced networks."

  FUNCTIONAL REQUIREMENTS

  The system must:

  - build probe packets with controlled flow identifiers to discover routers on each hop.
  - apply a multipath discovery solution to find all valid paths through load-balanced networks.
  - allow the user to have control over probes per interface.

  NON-FUNCTIONAL REQUIREMENTS

  The system must:

  - support reliable multipath exploration in a controlled network environment.

- **User Story:** "As a user, I want to reliably test each tool in a controlled network environment to verify correct probing behaviour."

  FUNCTIONAL REQUIREMENTS

  The system must:

  - provide a controlled environment where each tool can produce accurate test results.

- offer a test setup where network behaviour is stable, predictable, and isolated from external routing changes.

NON-FUNCTIONAL REQUIREMENTS

The system must:

- ensure repeatability of results.
- be easy for users to set up and operate with minimal configuration.

3.5 Design

At its core, Glasgow Traceroute is built to be modular in both its interface and system design. This approach lent itself well to the iterative development process described in the methodology as each application could be implemented incrementally while reusing the same underlying packet handling infrastructure.

3.5.1 Interface Design

The choice of a command-line interface (CLI) mirrors that of the tools upon which Glasgow Traceroute is built. An easy-to-use CLI is familiar to most network engineers and is well suited to the environments in which network measurement tools are typically used. For example, when testing within Mininet or other headless environments, a graphical interface would offer little benefit. The direct textual output provided by the CLI supported development and simplified verification of the probing behaviour during testing. The available commands and options provided by the CLI can be viewed using the built-in help command as shown below:

```
Usage: glasgow-traceroute [OPTIONS] <TOOL> <PROBE_TYPE> <DESTINATION>

Arguments:
  <TOOL>         Tool to run
                 [possible values: ping, traceroute, mda]

  <PROBE_TYPE>   Probe protocol
                 [possible values: icmp, udp]

  <DESTINATION>  Target host as an IPv4 address (e.g. 8.8.8.8)

Options:
  --port <PORT>        UDP destination port for 'ping udp'
                       (default: 33434)

  --topology <FILE>    Path to topology YAML file

  --mda-probes <N>     Number of probes per discovery round (default: 7)

  -h, --help           Print help
```

In addition to the CLI, a lightweight topology visualiser was developed to assist when testing within controlled environments where the network topology is already known. This tool allows the measured routes to be compared against the expected topology, providing a convenient way to verify probing behaviour during experimentation.

Although the primary interface of Glasgow Traceroute is a CLI, the system was designed such that the probing logic is independent from the presentation layer. As a result, alternative interfaces could easily be developed without modifying the underlying measurement functionality. The internal structure that enables this separation between interface and probing logic is described in the following section.

3.5.2 System Design

While Figure 5 provides a detailed view of the system's primary components. A higher level overview and additional perspectives are presented in Appendix A (Figures 12 and 13).

- **Network Measurement Tool**
  - **Interface**

    Command Line Interface (CLI)

    · The command line interface is the entry point of the system. Its purpose is to parse user input and dispatch execution to the appropriate network application.

    Topology Visualiser

    · The topology visualiser is a supporting component used for measuring Traceroute paths on known networks. It provides a textual representation of the network structure highlighting the path returned by Traceroute. Presenting the topology as ASCII output ensures this feature is usable in situations where graphical interfaces may not be available.

  - **Applications**

    All three applications are facilitated by a shared packet handling infrastructure responsible for constructing probe packets and matching responses.

    Ping

    · Ping is the most fundamental application of which the others are derived from. Its purpose is to measure reachability and round trip time between the source host and destination using ICMP or UDP probes.

    Traceroute

    · Traceroute builds upon the same probing infrastructure to discover the sequence of routers between the source and destination.

    Multipath Detection Algorithm (MDA

    · The Multipath Detection Algorithm extends Traceroute functionality by exploring multiple possible paths between the source and destination in load-balanced networks.

  - **Packet Handling**

    Headers

- The headers module is responsible for crafting packet headers (e.g., ICMP and UDP), ensuring they conform to the expected RFC standards.

Packet Parser

- The packet parser processes incoming responses, extracting relevant header fields to accurately correlate ICMP and ICMP error messages with their corresponding probe packets.

- **Test Network**
  - **Simulation**

    Networks are simulated using a software-defined network (SDN) application which allows controlled experimentation with custom network topologies.

  - **Routing**

    Routers use Free Range Routing (FRR) with OSPF and multipath forwarding to create multiple paths between endpoints.

  - **Endpoints**

    Hosts are connected to the network via switches and reach the mesh through a default gateway.

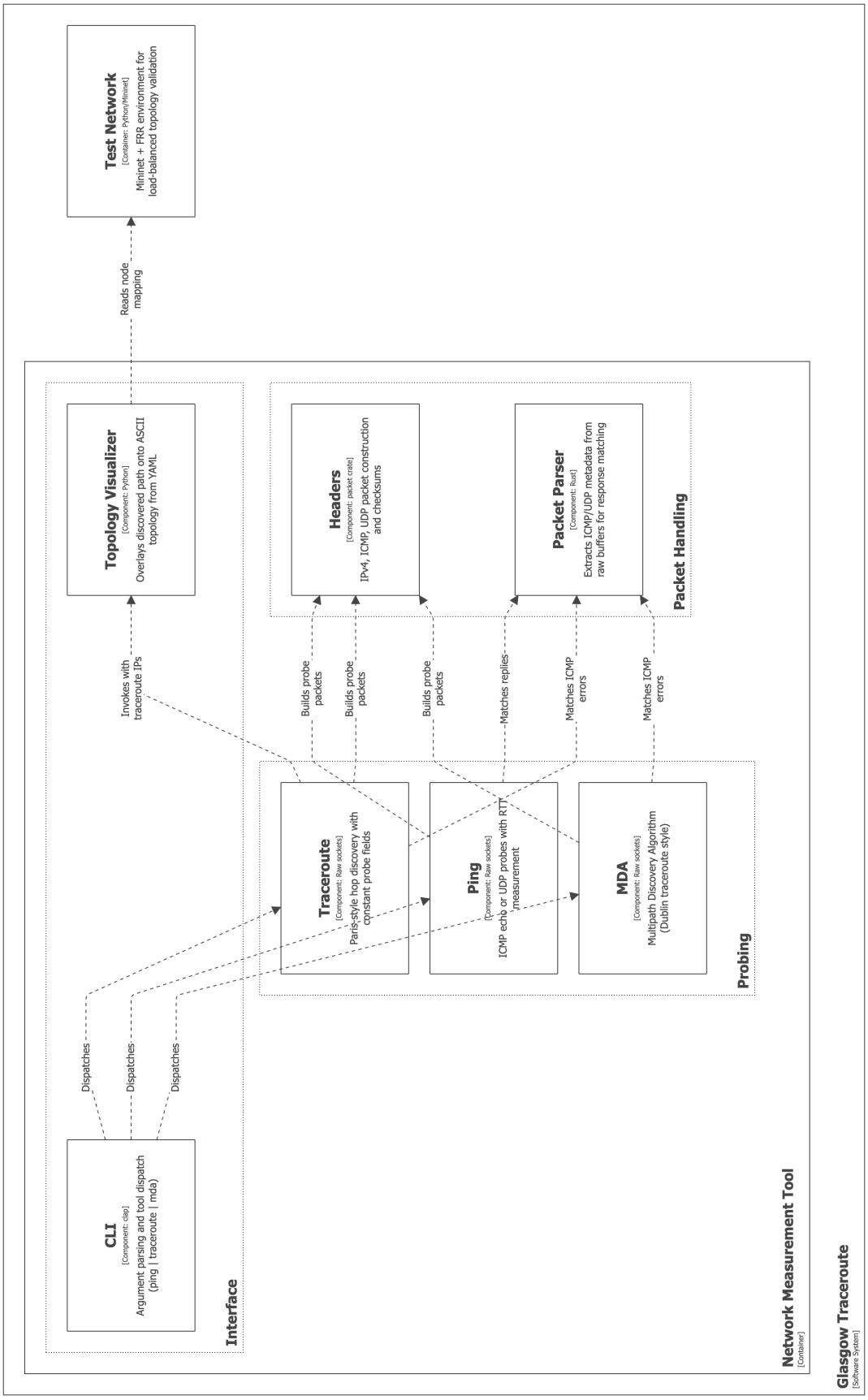
Glasgow Traceroute
[Software System]
Network Measurement Tool
[Container]
Interface
CLI
[Component: clap]
Argument parsing and tool dispatch
(ping | traceroute | mda)
Topology Visualizer
[Component: Python]
Overlays discovered path onto ASCII
topology from YAML
Probing
Traceroute
[Component: Raw sockets]
Paris-style hop discovery with
constant probe fields
Ping
[Component: Raw sockets]
ICMP echo or UDP probes with RTT
measurement
MDA
[Component: Raw sockets]
Multipath Discovery Algorithm
(Dublin traceroute style)
Packet Handling
Headers
[Component: packet crate]
IPv4, ICMP, UDP packet construction
and checksums
Packet Parser
[Component: Rust]
Extracts ICMP/UDP metadata from
raw buffers for response matching
Test Network
[Container: Python/Mininet]
Mininet + FRR environment for
load-balanced topology validation
Reads node
mapping
Dispatches
Dispatches
Dispatches
Invokes with
traceroute IPs
Builds probe
packets
Builds probe
packets
Builds probe
packets
Matches replies
Matches ICMP
errors
Matches ICMP
errors

Figure 5

4 Product

4.1 Implementation Overview

This section describes the technical implementation of Glasgow Traceroute in detail. The system was primarily written in the Rust programming language due to its strong memory safety guarantees and the suitability of its packages for systems-level networking applications. Python and the Bash scripting language also played a supporting role in the implementation, primarily for automating Linux environment setup and configuring the software-defined network used for testing.

Development of the core network measurement tool was carried out using Rust's Cargo build system, alongside a selection of third-party crates. Notable dependencies include the socket2 crate, which was used to create and manage raw sockets; the clap crate, which provided a structured approach to building the command line interface(CLI); and the packet crate which computed protocol checksums ensuring the packets conformed to the relevant protocol specifications.

The project was structured as an extensible library in order to maintain a clear separation of concerns through Rust's module system. Version control was managed using the university's GitLab server, enabling iterative development and tracking of implementation changes throughout the project lifecycle. The implementation of these modules and their role within the system architecture are described below.

4.2 Packet Handling

4.2.1 Headers

The headers module provides the packet structures used by each probing application. It implements functionality for IPv4, ICMP and UDP headers, along with builder types that simplify construction. The wrapper enum, `TransportHeader`, is used such that application components can handle probes through a shared interface. Each header was adapted from its corresponding Request For Comment (RFC) specification.

A notable detail regarding the IPv4 header is that it must be serialised differently depending on the operating system. On Linux, all multi-byte fields are written in network byte order, as expected for raw IP packets. On macOS, however, certain fields such as the total length and the

flags/fragment-offset word must be written in host byte order to satisfy the raw socket interface. This is handled by the use of compiler flags in the `to_byte_array` function as shown below:

```
#[cfg(target_os = "macos")]
buf.extend_from_slice(&self.total_length.to_ne_bytes());

#[cfg(target_os = "linux")]
buf.extend_from_slice(&self.total_length.to_be_bytes());
```

The ICMP header follows the structure defined for Echo Request messages and includes the type, code, checksum, identifier, and sequence number fields. During serialisation the checksum is calculated over the header and payload using the `packet` crate. For Traceroute probes, the sequence number is incremented while the identifier is decremented to maintain a constant checksum, following the approach described by Paris Traceroute.

Furthermore, the UDP header contains the source port, destination port, length, and checksum fields. In the multipath probing implementation the source port is used as the flow identifier ϕ . To keep the UDP checksum constant across flows, the destination port is adjusted as a function of ϕ so that the sum of the two ports remains fixed. This preserves a stable checksum while still allowing the flow identifier to vary.

Together, these components provide the low-level packet construction required by the probing tools while ensuring that the generated packets conform to the relevant protocol specifications

4.2.2 Packet Parser

The packet parser reads incoming packet bytes and extracts the information needed to match replies to sent probes. It supports ICMP Echo Replies as well as ICMP error messages carrying embedded UDP or ICMP headers, and performs checks on packet structure and expected field values before accepting a response.

4.3 Probing

4.3.1 Ping

Glasgow Traceroute's Ping application supports both ICMP echo probes and UDP probes allowing behaviour consistent with the original version by Muuss.

When a Ping instance is initialised using the `Ping::new` method, user input from the command-line interface determines the transport header to be built. Although ICMP is formally a network-layer control protocol rather than a transport protocol, it is treated alongside UDP as a probing protocol for implementation convenience. Based on this selection, the appropriate header structure is prepared while the IPv4 header and raw socket required for probe transmission are also initialised. Raw sockets allow full control over the construction of packet headers, which is essential for implementing techniques such as Paris Traceroute that require precise manipulation of protocol fields. Consequently, raw socket functionality was first implemented and validated within the Ping application before being reused by the more complex Traceroute and MDA components, in line with the iterative development methodology described earlier.

Probes are sent via function calls to `Ping::send_ping` where the transport header is converted into a byte representation along with the payload. For ICMP probes the sequence number is incremented for each transmission, allowing replies to be matched with their corresponding requests, while the identifier remains constant for the duration of the session. In the case of UDP probes, a randomly selected source port within the ephemeral range is used so that the resulting ICMP error responses can be associated with the original probe. Prior to transmission of the constructed probe, protocol checksums for UDP and ICMP are handled via the `packet` crate.

Once transmitted, a receive buffer is initialised and the function enters a loop until a matching packet is received. The buffer is wrapped within an `unsafe` block as the `recv_from` function requires uninitialised memory for incoming packet data. A second `unsafe` block is used to interpret the bytes written by the socket as a byte slice. By keeping the buffer and slice creation in an `unsafe` block the rest of the program stays in memory safe Rust. Matched packets are then returned to the client wrapped in a `PingResult` struct containing the round-trip time and packet statistics. The continuous probing and interrupt handling are managed by the client, which repeatedly invokes `Ping::send_ping` until the user terminates the process.

4.3.2 Traceroute

Traceroute follows a similar flow to Ping, where probe packets are constructed using the header modules, transmitted to the destination, and responses are parsed and matched with the corresponding probe. However, instead of repeatedly probing the destination host, Traceroute incrementally increases the Time To Live (TTL) between each probe to reveal the hop by hop

path as described in earlier sections.

To ensure accurate measurements in load balanced networks, this implementation follows the principles of Paris Traceroute by keeping flow identifying fields constant across probes. For UDP probes the source and destination ports remain fixed for the duration of the trace. For ICMP probes the identifier remains constant while the sequence number increments with each hop, allowing responses to be associated with the correct probe while preserving a consistent flow identifier.

The tracing process continues until either the destination host responds or the predefined maximum TTL is reached. Each hop is returned as a `HopResult` containing the TTL, responding router address and measured round-trip time. These results are collected into a sequence that can be formatted by the command-line interface or passed to the topology visualisation tool.

4.3.3 MDA

The multipath Detection Algorithm extends Traceroute to discover all forwarding paths in load balanced networks by varying the flow identifier of probes so that they may be routed along different next hops.

The implementation is broadly consistent with the approach proposed by Augustin et al. , but adopts a simplified stopping rule similar to that of Dublin Traceroute. Instead of using a statistical confidence threshold, the user specifies a fixed number of initial flows to generate. At each hop, the algorithm identifies the set of responding interfaces and the flow identifiers that reached them. For the first hop, probes are sent using the user generated flow identifiers. For subsequent hops, the algorithm reuses the flow identifiers that successfully reached the previous hop, allowing it to explore how those flows propagate through the network. This hop-by-hop exploration gradually reveals the set of distinct paths between the source and the destination.

To maintain consistency with the original MDA algorithm, the variables used in the implementation map to those defined in the paper. The correlation between this notation and the variable names is documented as comments in the source code and summarised in Appendix A. In the original design, the algorithm is presented as pseudo-code, with key aspects of state management, flow exploration and stopping conditions left implicit. Translating this into a concrete implementation was non-trivial. Glasgow Traceroute's fixed flow approach

provides a clearer implementation while remaining faithful to the original paper. This is further supported by the evaluation in Chapter 5, where non-terminating behaviour is observed using Paris Traceroute.

During development, an ambiguity was identified in Algorithm 2 of the original paper. In the step where the successor set is updated, the paper specifies $\hat{S}_r \leftarrow \hat{S}_r \cup \{r\}$, where r denotes the current router and s the newly discovered successor interface. Adding r would incorrectly update the set with the current node rather than the next hop. The implementation therefore uses $\hat{S}_r \leftarrow \hat{S}_r \cup \{s\}$.

4.4 Interface

The client interface implemented in `main.rs` serves as the single entry point for Glasgow Traceroute. Its responsibilities are parsing command-line arguments using the `clap` library and dispatching execution to the appropriate user selected application.

For Traceroute, the client optionally invokes a Python based topology visualisation. The script loads a pre-defined topology from a YAML file and constructs a directed graph using the `NetworkX` library. Discovered hops are mapped to nodes within the graph and rendered as ASCII art using the `phart` library.

4.5 Verification & Validation

4.5.1 Network Test Environment

To validate each of the measurement tools, a controlled virtual network environment was developed. This was achieved by using a software defined network (SDN) called Mininet. As Mininet requires Ubuntu 22.04, a virtual machine provisioned through Multipass ensured that it could be run on multiple operating systems. This approach along with the automated bash scripts used to install dependencies satisfies both functional and non-functional requirement of having a reproducible test environment.

Mininet runs hosts, switches, and routers within isolated Linux network namespaces, allowing low-level control over network behaviour through system configuration parameters such as `sysctl`. This enables features such as IP forwarding, multipath hashing, and ICMP response behaviour to be configured on a per-router basis. This made it possible to reproduce the load-

balanced topologies described in the literature and to evaluate measured paths.

Three load-balanced test topologies were built as defined in their respective `mn_topo.py` python files. The first implements a simple diamond structure as shown in Figure 6, while Figure 7 introduces nested load-balancing to evaluate behaviour across multiple stages of path divergence. In this topology, different multipath hashing policies were applied to each load balancer to avoid traffic polarisation and ensure flows were distributed across distinct paths. A third topology shown in Figure 8 was developed to provide an additional controlled scenario for validating and troubleshooting the multipath implementation. As the structure and routing configuration of each topology were predefined, the correctness of the observed probing results could be verified directly. In each Figure 6, 7, 8 circles represent routers, squares represent switches and hosts, and lines represent links. Pink markers indicate router interfaces, with IP addresses shown next to each interface.

4.5.2 Unit Testing

Unit testing was limited to the packet parsing module, as it operates on raw byte data and can be tested deterministically. The main measurement tools depend on network behaviour and were therefore validated through using the virtual network environment.

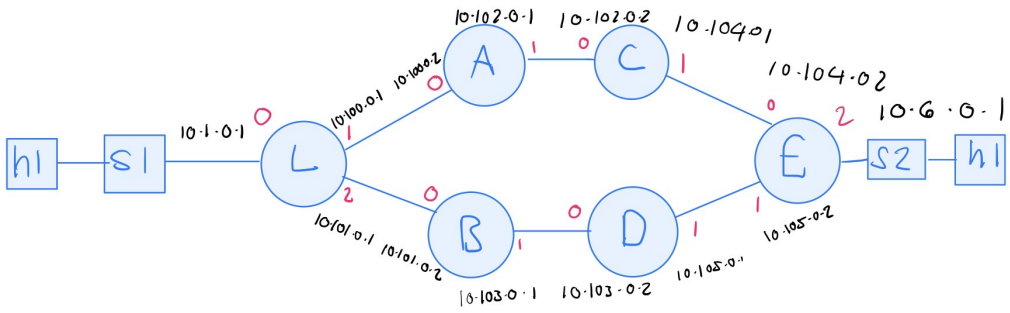

Figure 6: Simple diamond topology

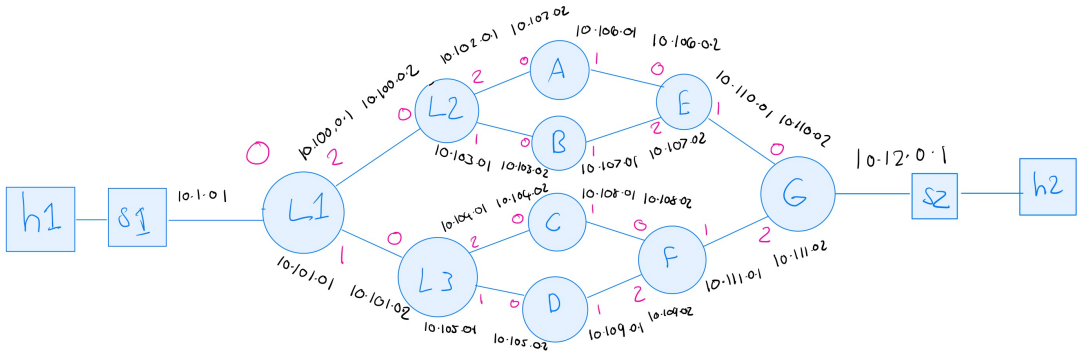

Figure 7: Nested load-balanced topology

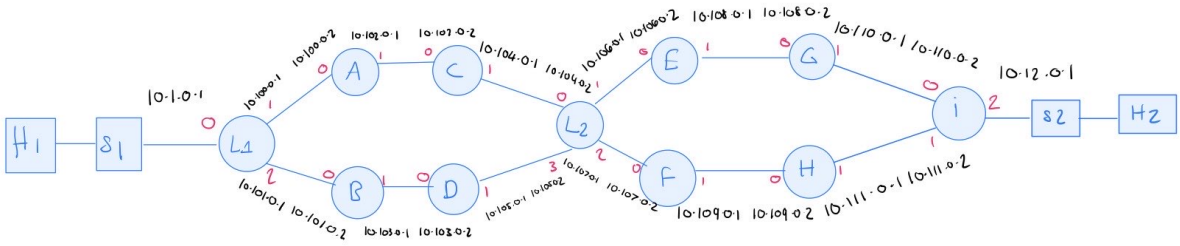

Figure 8: Repeated diamond topology

5 Results & Evaluation

This section shows the results obtained from testing each measurement tool in the created Test Network.

5.1 Results of Evaluation

5.1.1 Ping

The Ping implementation was evaluated by sending ICMP and UDP probes to a target host `h2` within the controlled network environment shown in Figure 6. The results showed consistent round-trip times with no packet loss under normal conditions, indicating that probe packets were correctly constructed and responses were successfully matched to their corresponding requests. This is shown in Figure 9.

```
mininet> h1 /home/ubuntu/git/glasgow-traceroute/target/debug/glasgow-traceroute ping icmp h2
PING 10.6.0.251:
Sent ICMP echo request, received ICMP echo reply: 64 bytes from 10.6.0.251: icmp_seq=1 time=4.105 ms
Sent ICMP echo request, received ICMP echo reply: 64 bytes from 10.6.0.251: icmp_seq=2 time=2.452 ms
Sent ICMP echo request, received ICMP echo reply: 64 bytes from 10.6.0.251: icmp_seq=3 time=0.565 ms
Sent ICMP echo request, received ICMP echo reply: 64 bytes from 10.6.0.251: icmp_seq=4 time=0.243 ms
Sent ICMP echo request, received ICMP echo reply: 64 bytes from 10.6.0.251: icmp_seq=5 time=0.263 ms
^C
--- 10.6.0.251 ping statistics ---
5 packets transmitted, 5 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 0.243/1.526/4.105/1.528 ms
mininet> h1 /home/ubuntu/git/glasgow-traceroute/target/debug/glasgow-traceroute ping udp h2
PING 10.6.0.251:
Sent UDP request, received ICMP reply: 92 bytes from 10.6.0.251: time=5.545 ms
Sent UDP request, received ICMP reply: 92 bytes from 10.6.0.251: time=4.576 ms
Sent UDP request, received ICMP reply: 92 bytes from 10.6.0.251: time=2.105 ms
Sent UDP request, received ICMP reply: 92 bytes from 10.6.0.251: time=2.343 ms
Sent UDP request, received ICMP reply: 92 bytes from 10.6.0.251: time=2.931 ms
^C
--- 10.6.0.251 ping statistics ---
5 packets transmitted, 5 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 2.105/3.500/5.545/1.338 ms
```

Figure 9: Ping results showing consistent RTT measurements with no packet loss.

This confirms that the underlying packet handling and checksum computation are functioning as expected, satisfying the requirement for accurate reachability testing and response parsing.

5.1.2 Traceroute

Traceroute's results were evaluated using the topology in Figure 8, which contains two stages of load balancing. Figure 10 shows its results in comparison to Jacobson's Classic Traceroute.

```
mininet> h1 /home/ubuntu/git/glasgow-traceroute/target/debug/glasgow-traceroute traceroute icmp h2 --topology topology.yaml
traceroute to 10.12.0.251 (10.12.0.251), 30 hops max
 1  10.1.0.1  1.509 ms
 2  10.100.0.2  1.997 ms
 3  10.102.0.2  0.570 ms
 4  10.104.0.2  1.078 ms
 5  10.106.0.2  0.545 ms
 6  10.108.0.2  1.808 ms
 7  10.110.0.2  0.892 ms
 8  10.12.0.251  0.642 ms


===TOPOLOGY===

Path: h1 -> l1 -> a -> c -> l2 -> e -> g -> i -> h2

a: 10.100.0.2
b: 10.101.0.2
c: 10.102.0.2
d: 10.103.0.2
e: 10.106.0.2
f: 10.107.0.2
g: 10.108.0.2
h: 10.109.0.2
h1: 10.1.0.251
h2: 10.12.0.251
i: 10.12.0.1
l1: 10.1.0.1
l2: 10.104.0.2
s1: null
s2: null

       [h1]
        ↓
       [s1]
        ↓
       [l1]
        ↓
    [a]←+→[b]
     ↓     ↓
    [c]   [d]
     ↓     ↓
     +→[l2]←+
        ↓
    [e]←+→[f]
     ↓     ↓
    [g]   [h]
     ↓     ↓
     +→[i]←+
        ↓
       [s2]
        ↓
       [h2]



mininet> h1 traceroute h2
traceroute to 10.12.0.251 (10.12.0.251), 30 hops max, 60 byte packets
 1  10.1.0.1 (10.1.0.1)  2.349 ms  2.307 ms  2.228 ms
 2  10.101.0.2 (10.101.0.2)  2.294 ms 10.100.0.2 (10.100.0.2)  2.302 ms  2.307 ms
 3  10.102.0.2 (10.102.0.2)  2.311 ms  2.314 ms 10.103.0.2 (10.103.0.2)  2.318 ms
 4  10.104.0.2 (10.104.0.2)  2.320 ms  2.322 ms  2.324 ms
 5  10.107.0.2 (10.107.0.2)  2.335 ms 10.106.0.2 (10.106.0.2)  2.338 ms  2.341 ms
 6  10.108.0.2 (10.108.0.2)  2.353 ms  2.042 ms  1.998 ms
 7  10.111.0.2 (10.111.0.2)  2.013 ms 10.110.0.2 (10.110.0.2)  1.984 ms 10.111.0.2 (10.111.0.2)  2.001 ms
 8  10.12.0.251 (10.12.0.251)  3.827 ms  3.844 ms  3.844 ms
```

Figure 10: Comparison of Glasgow Traceroute and Classic Traceroute outputs in a load-balanced topology.

The Glasgow Traceroute implementation produced the following stable path:

$$h1 \rightarrow l1 \rightarrow a \rightarrow c \rightarrow l2 \rightarrow e \rightarrow g \rightarrow i \rightarrow h2$$

The original Paris Traceroute tool produces an identical path on this topology.

Whereas, the Classic Traceroute output shows an impossible path identified by the two transitions $b \rightarrow c$ and $f \rightarrow g$:

$$h1 \rightarrow l1 \rightarrow b \rightarrow c \rightarrow l2 \rightarrow f \rightarrow g \rightarrow i \rightarrow h2$$

These results confirm that the implemented Traceroute maintains a constant flow identifier between probes, ensuring consistent forwarding behaviour through load balancers and avoiding incorrect path construction.

5.1.3 MDA

The MDA implementation was evaluated using the nested load balanced topology shown in Figure 7. Figure 11 shows the results of the implemented MDA in comparison to existing multipath tracing approaches.

```
mininet> h1 /home/ubuntu/git/glasgow-traceroute/target/debug/glasgow-traceroute mda udp h2
mda traceroute to 10.12.0.251 (10.12.0.251), 30 hops max (UDP)
Path 1:
   1  10.1.0.1
   2  10.100.0.2
   3  10.102.0.2
   4  10.106.0.2
   5  10.110.0.2
   6  10.12.0.251

Path 2:
   1  10.1.0.1
   2  10.100.0.2
   3  10.103.0.2
   4  10.107.0.2
   5  10.110.0.2
   6  10.12.0.251

Path 3:
   1  10.1.0.1
   2  10.101.0.2
   3  10.104.0.2
   4  10.108.0.2
   5  10.111.0.2
   6  10.12.0.251

Path 4:
   1  10.1.0.1
   2  10.101.0.2
   3  10.105.0.2
   4  10.109.0.2
   5  10.111.0.2
   6  10.12.0.251
mininet> h1 paris-traceroute -a mda -U h2
mda to 10.12.0.251 (10.12.0.251), 30 hops max, 30 bytes packets
0 None -> 10.1.0.1 [{ 0*1, 0*2, 0*3, 0*4, 0*5, 0*6, 0!7, 0!8 } -> {  1*6, 1*1, 1*2, 1*3, 1*4, 1*5, 1*9, 1*10 }]
1 10.1.0.1 -> 10.100.0.2 [{ 1*6, 1*1, 1*2, 1*3, 1*4, 1*5, 1*9, 1*10, 1*11, 1!12, 1*13, 1*14, 1!15, 1!16, 1!17 } -> {  2*6, 2*2, 2*3, 2*9, 2*10, 2*11, 2?18, 2?19 }]
1 10.1.0.1 -> 10.101.0.2 [{ 1*6, 1*1, 1*2, 1*3, 1*4, 1*5, 1*9, 1*10, 1*11, 1!12, 1*13, 1*14, 1!15, 1!16, 1!17 } -> {  2*1, 2*4, 2*5, 2*13, 2*14, 2?20, 2?21, 2?22 }]
2 10.100.0.2 -> 10.102.0.2 [{ 2*6, 2*2, 2*3, 2*9, 2*10, 2*11, 2?18, 2?19, 2?23, 2?24, 2?25, 2?26, 2?27, 2?28, 2?29 } -> {  3 2, 3 10, 3 11 }]
2 10.101.0.2 -> 10.104.0.2 [{ 2*1, 2*4, 2*5, 2*13, 2*14, 2?20, 2?21, 2?22, 2?30, 2?31, 2?32, 2?33, 2?34, 2?35, 2?36 } -> {  3 14 }]
2 10.100.0.2 -> 10.103.0.2 [{ 2*6, 2*2, 2*3, 2*9, 2*10, 2*11, 2?23, 2?24, 2?25, 2?26, 2?27, 2?28, 2?29, 2*18, 2?37, 2*20 } -> {  3 6, 3 3, 3 9, 3 18, 3 20 }]
2 10.101.0.2 -> 10.104.0.2 [{ 2*1, 2*4, 2*5, 2*13, 2*14, 2?30, 2?31, 2?32, 2?33, 2?34, 2?35, 2?36, 2*19, 2*21, 2*22 } -> {  3 14, 3 22 }]
^Cevent not yet handled
Lattice:
None -> [ 10.1.0.1 ]
10.1.0.1 -> [ 10.100.0.2, 10.101.0.2 ]
10.100.0.2 -> [ 10.103.0.2, 10.102.0.2 ]
10.103.0.2
10.102.0.2
10.101.0.2 -> [ 10.105.0.2, 10.104.0.2 ]
10.105.0.2
10.104.0.2
```

Figure 11: Comparison of MDA results across different Traceroute implementations in a load-balanced topology.

The Glasgow Traceroute implementation successfully returned all four paths using the default 7 initial flows:

Path 1: $h1 \rightarrow l1 \rightarrow l2 \rightarrow a \rightarrow e \rightarrow g \rightarrow h2$

Path 2: $h1 \rightarrow l1 \rightarrow l2 \rightarrow b \rightarrow e \rightarrow g \rightarrow h2$

Path 3: $h1 \rightarrow l1 \rightarrow l3 \rightarrow c \rightarrow f \rightarrow g \rightarrow h2$

Path 4: $h1 \rightarrow l1 \rightarrow l3 \rightarrow d \rightarrow f \rightarrow g \rightarrow h2$

These paths match the expected structure of the topology, demonstrating that the implementation correctly explores all branches introduced by successive load balancers and reconstructs complete end-to-end paths.

For comparison, Paris Traceroute in MDA mode identified the available interfaces at each load balancing stage but did not terminate cleanly and required manual interruption. While it revealed the branching structure of the network, it did not produce a complete set of end-to-end paths.

Dublin Traceroute was also evaluated using 50 flows. Its output exposed the same branching behaviour and produced several complete flow-specific paths. However, many flows terminated prematurely or resulted in incomplete paths, making it difficult to directly infer the full set of unique end-to-end routes.

6 Conclusion

This project successfully achieved the goal of implementing an extensible and memory-safe version of Paris Traceroute. It addressed the limitations of traditional Traceroute in load-balanced networks by maintaining control over flow-identifying fields and producing consistent and accurate path measurements. The system was designed as a modular and reusable library, supporting multiple probing strategies including Ping, Traceroute, and a simplified Multipath Detection Algorithm.

This short section examines the results of the project, reflecting on the challenges encountered during development and the limitations of the implementation.

Overall, this dissertation has achieved its aim of implementing a reliable Paris Traceroute application using Rust, a modern systems programming language with improved memory safety. By reducing the likelihood of memory-related vulnerabilities, this work contributes to more secure network measurement practices and establishes a foundation for future research into network path optimisation.

6.1 Interpreting the Results

The results show that the implementation produces correct behaviour in load-balanced networks. The Traceroute component avoided the inconsistencies observed in Classic Traceroute and returned paths that matched the known topology, while the MDA implementation successfully identified multiple valid end-to-end paths. Together, these outcomes confirm that the system satisfies the core requirement of accurate and multipath-aware network measurement.

6.2 Challenges and Limitations

A significant portion of the project was spent resolving issues within the virtual network environment rather than extending functionality such as TCP or IPv6 support. These issues were not related to Mininet itself, but rather with the default Linux kernel `sysctl` settings. In particular, the `icmp_errors_use_inbound_ifaddr` setting was the most significant. When disabled (the default), ICMP error messages are sent using the primary address of the outgoing interface rather than the interface that received the original packet. This resulted in responses

appearing to originate from unexpected addresses, leading to incorrect probe matching during testing. Resolving this issue was necessary to ensure correct behaviour of the implementation.

A further limitation is that the evaluation was conducted entirely within a controlled software defined network. While this worked well for validating against known topologies, it wasn't possible to assess how it fared in real-world network environments. As a result, the findings do not guarantee equivalent behaviour across diverse or unpredictable configurations.

6.3 Future Work

Future work could focus on the following ideas:

- Improving the scalability and efficiency of the implementation. This would involve introducing concurrency to allow probes to be sent in parallel.
- Implementing the stopping condition as described in the original paper using a probabilistic stopping point instead of a fixed number of flow generation.
- Extending the work to other network measurement tools, such as implementing an operating system fingerprinting tool in a memory-safe language.

References

[1] J. D. Day and H. Zimmermann, "The osi reference model," *Proceedings of the IEEE*, vol. 71, no. 12, pp. 1334–1340, 1983.

[2] "Internet Protocol." RFC 791, Sept. 1981.

[3] "Internet Control Message Protocol." RFC 777, Apr. 1981.

[4] "Transmission Control Protocol." RFC 793, Sept. 1981.

[5] "User Datagram Protocol." RFC 768, Aug. 1980.

[6] M. Muuss, "The story of the ping program." `https://ftp.arl.army.mil/~mike/ping.html`, 1983. Originally distributed with BSD Unix.

[7] V. Jacobson, "Traceroute." `ftp://ftp.ee.lbl.gov/traceroute.tar.gz`, 1989. Lawrence Berkeley Laboratory.

[8] J. Moy, "OSPF Version 2." RFC 2328, Apr. 1998.

[9] B. Augustin, X. Cuvellier, B. Orgogozo, F. Viger, T. Friedman, M. Latapy, C. Magnien, and R. Teixeira, "Avoiding traceroute anomalies with paris traceroute," in *Proceedings of the 6th ACM SIGCOMM conference on Internet measurement*, pp. 153–158, 2006.

[10] B. Augustin, T. Friedman, and R. Teixeira, "Multipath tracing with paris traceroute," in *2007 Workshop on End-to-End Monitoring Techniques and Services*, pp. 1–8, IEEE, 2007.

[11] A. Barberio, "Dublin traceroute."

[12] MSRC Team, "We need a safer systems programming language." `https://www.microsoft.com/en-us/msrc/blog/2019/07/we-need-a-safer-systems-programming-language/`, July 2019. Microsoft Security Response Center Blog.

[13] W. Bugden and A. Alahmar, "The safety and performance of prominent programming languages," *International Journal of Software Engineering and Knowledge Engineering*, vol. 32, no. 05, pp. 713–744, 2022.

A Appendix A - Detailed Specification and Design

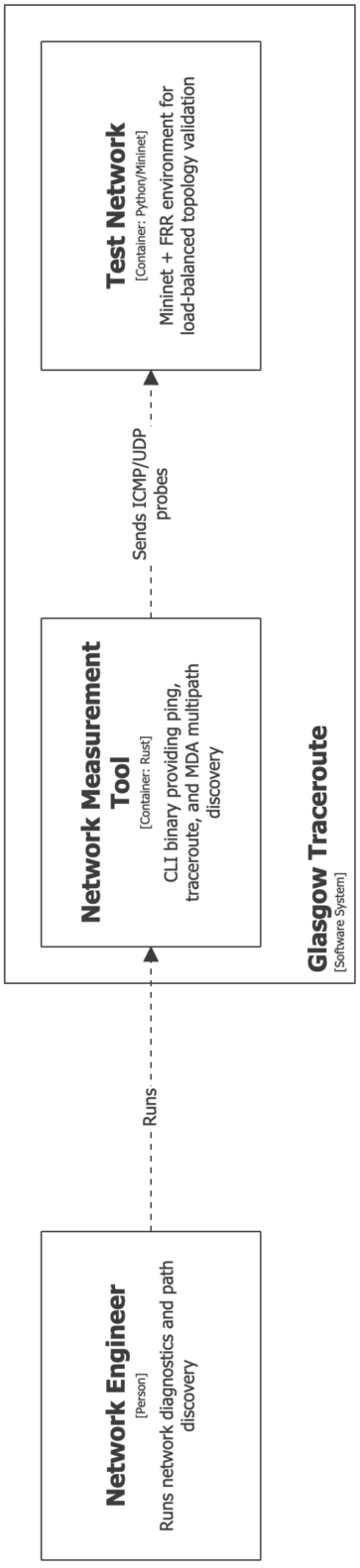

Figure 12

Table 2: Algorithm 1: Symbol mapping

Descriptive name	**Symbol**
Previous-hop interface set (initial)	$\hat{R}_{h_{\min}-1} \leftarrow \{0\}$
Current-hop interface set	$\hat{R}_h$
Current interface being expanded	r
Next-hop interface set from r	$\hat{S}_r$
Previous-hop flows reaching r	$F_{h-1,r}$
Current-hop flows reaching interface s	$F_{h,s}$
Per-packet behavior at interface r	π_r
Minimum hop bound	$h_{\min}$
Maximum hop bound	$h_{\max}$
Global confidence parameter	α_{all}
Next-hop confidence parameter	α_{nexthops}

Table 3: Algorithm 2: Symbol mapping

Descriptive name	**Symbol**
Probe counter / progress index	k
Next-hop interface set from interface r	$\hat{S}_r$
Current guess for number of next hops	$n \leftarrow \lvert\hat{S}_r\rvert + 1$
Flow set that reached interface r	$F_{h-1,r}$
Flow set reaching successor interface s at hop h	$F_{h,s}$
Current successor interface discovered by a probe	s
Current probe flow identifier	ϕ
Hop index / TTL being probed	h
Source interface flag condition	$r = \text{source}$

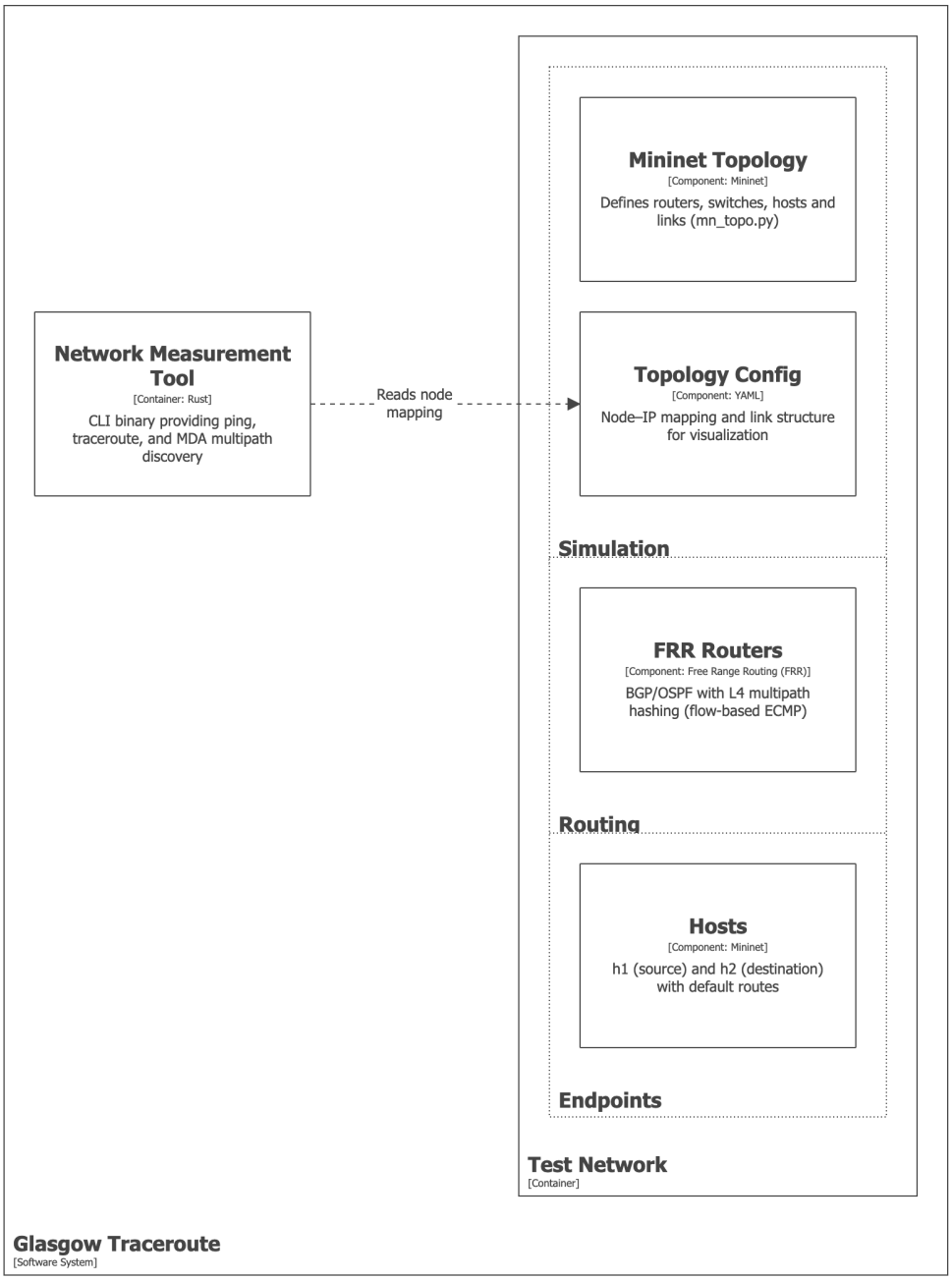
Glasgow Traceroute
[Software System]
Network Measurement Tool
[Container: Rust]
CLI binary providing ping, traceroute, and MDA multipath discovery
Reads node mapping
Test Network
[Container]
Simulation
Mininet Topology
[Component: Mininet]
Defines routers, switches, hosts and links (mn_topo.py)
Topology Config
[Component: YAML]
Node–IP mapping and link structure for visualization
Routing
FRR Routers
[Component: Free Range Routing (FRR)]
BGP/OSPF with L4 multipath hashing (flow-based ECMP)
Endpoints
Hosts
[Component: Mininet]
h1 (source) and h2 (destination) with default routes

Figure 13